MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD
Department of Electronics and Communication Engineering

DIGITAL SIGNAL PROCESSING
PROJECT REPORT

Secure PDF Document Authentication Using Cryptographic Hashing and QR Codes

Submitted by
Prajapati Vishnu Munna — 20244127
Pranay Mathurkar — 20244129
Pranav Ranjan — 20244128
Payel Sardar — 20244126

Academic Year: 2026
Project stage: Completed prototype

Supervisor: [INFORMATION REQUIRED]

Abstract

Digital documents are frequently altered after issuance, while conventional full-file hashing is too sensitive to harmless PDF container changes and cannot explain where a visual modification occurred. This project implements a two-stage authentication system for PDF documents. During securing, the first page is rendered deterministically at 150 DPI in RGB format, and the raw pixel buffer is hashed with SHA-256. The digest is encoded in a high-error-correction QR code and appended as a second A4 authentication page. During verification, the QR payload is decoded, the candidate first page is rendered using the same canonical procedure, and the two digests are compared in constant time.

The implementation extends exact cryptographic verification with an independent digital-signal-processing forensic pipeline. When a stored canonical reference is available, the system computes SSIM, MSE, PSNR, FFT similarity, DCT similarity, edge similarity, and localized tampering regions. It returns visual evidence including difference images, binary masks, heatmaps, FFT spectra, and bounding-box overlays. The repository contains FastAPI endpoints, a React/Vite web portal, automated service tests, and an end-to-end workflow that exercises authentic, modified, and invalid documents. The system is therefore best characterized as a completed working prototype rather than a statistically benchmarked production system; quantitative claims are limited to the included tests and supplied screenshots.

Keywords: PDF forensics, canonical rasterization, SHA-256, QR code, FFT, DCT, SSIM, tamper localization.

Acknowledgement

The authors acknowledge the Department of Electronics and Communication Engineering, Motilal Nehru National Institute of Technology Allahabad, for providing the academic context for this Digital Signal Processing project. The report was prepared from the supplied reference-report screenshots, the project repository, source code, automated tests, and the searched base-paper record listed in the references.

Table of Contents

This document uses Word heading styles so that a dynamic Table of Contents can be inserted or updated in Microsoft Word using References → Table of Contents → Update Table.

- Abstract
- Chapter 1: Introduction
- Chapter 2: Literature Review and Base Paper Analysis
- Chapter 3: Theoretical Background
- Chapter 4: Proposed System Architecture and Methodology
- Chapter 5: Implementation
- Chapter 6: Experimental Evaluation and Results
- Chapter 7: Discussion, Limitations and Future Work
- Chapter 8: Conclusion
- References
- Appendices

List of Figures

- Figure 1. Home page of the implemented authentication portal.
- Figure 2. Modified-document verdict and forensic telemetry.
- Figure 3. DSP visualization dashboard for localized tampering.

List of Tables

- Table 1. Project facts and evidence status.
- Table 2. Base paper and project comparison.
- Table 3. DSP configuration values.
- Table 4. Verification test matrix.

Abbreviations and Symbols

Term	Meaning
PDF	Portable Document Format
QR	Quick Response
SHA-256	Secure Hash Algorithm with 256-bit digest
FFT	Fast Fourier Transform
DCT	Discrete Cosine Transform
SSIM	Structural Similarity Index
MSE	Mean Squared Error
PSNR	Peak Signal-to-Noise Ratio
DPI	Dots per inch
DSP	Digital Signal Processing
RGB	Red, Green, Blue
API	Application Programming Interface

Chapter 1: Introduction

1.1 Background

Academic certificates, transcripts, identity documents, and institutional notices are commonly distributed as PDFs. A recipient needs to establish both provenance and visual integrity. A naïve SHA-256 hash over the complete PDF binary is exact, but PDF serialization can change when metadata, object ordering, compression, or cross-reference structures change. Conversely, a purely visual similarity score may tolerate or miss small but meaningful edits. The project addresses this tension by combining an exact canonical visual digest with interpretable DSP evidence.

1.2 Problem Statement

Design and implement a web-accessible system that seals the first page of an uploaded PDF, stores an authentication digest in an appended QR certificate, and verifies later submissions while identifying and visualizing visual alterations when an independent reference is available.

1.3 Objectives

- Create a reproducible first-page representation independent of volatile PDF binary structure.
- Generate a SHA-256 digest and embed it in a Level-H QR certificate.
- Provide REST APIs and a responsive React interface for securing and verifying PDFs.
- Add DSP-based forensic analysis using similarity, frequency, edge, and region-localization features.
- Test authentic, tampered, and invalid-document paths with automated tests.

1.4 Scope and Contributions

The implemented scope is a local or self-hosted prototype. It supports PDF upload, first-page extraction, QR-page synthesis, verification, persisted hash records and DSP references, and visual-result serving. It does not provide a trusted public key infrastructure, remote identity service, cryptographic digital signatures, multi-page canonical hashing, or a statistically representative benchmark dataset.

Project fact	Evidence / status
Institution and department	MNNIT Allahabad; Electronics and Communication Engineering (user-supplied brief)
Team	Four members and roll numbers supplied in the brief
Languages	Python; JavaScript/JSX
Core tools	FastAPI, PyMuPDF, hashlib, qrcode, OpenCV, NumPy, SciPy/scikit-image, React, Vite, Tailwind
Input	User-uploaded PDF documents; repository E2E test creates a synthetic transcript
Repository	https://github.com/Pranay-Mathurkar/Secure-PDF-Document-Authentication
Supervisor / publication venue	[INFORMATION REQUIRED]

1.5 Report Organization

Chapter 2 reviews the searched base paper and distinguishes its page-object strategy from this project. Chapter 3 presents the mathematical background. Chapters 4 and 5 describe the architecture and

implementation. Chapter 6 reports evidence from tests and supplied screenshots. Chapters 7 and 8 discuss limitations, future work, and conclusions.

Chapter 2: Literature Review and Base Paper Analysis

2.1 Base Paper Selected for Analysis

Because no complete base paper file was included with the supplied materials, the report uses the publicly indexed preprint “A Technique for the Detection of PDF Tampering or Forgery” by Gabriel Grobler, Sheunesu Makura, and Hein Venter, University of Pretoria. The record is available at arXiv:2507.00827 (2025): https://arxiv.org/abs/2507.00827. The paper is treated as a related baseline, not as evidence for results produced by this repository.

2.2 Base Paper Methodology

The searched description presents a structural PDF-forensics technique. It extracts page-level PDF objects or content streams, creates cryptographic hashes for those components during a protection phase, and compares later extracted objects against the stored baseline. A mismatch can identify an altered page or object and can cover structural changes that are not necessarily obvious in a rendered image. The approach is complementary to this project: it reasons about PDF structure, whereas this implementation deliberately authenticates the visual presentation of Page 1.

2.3 Comparison with the Implemented Project

Dimension	Base paper strategy	This project
Representation	PDF page objects/content streams	Rendered Page 1 raw RGB pixels
Primary integrity primitive	Hashes of extracted objects	SHA-256 over canonical pixels
Artifact placement	Baseline stored externally	Digest encoded in appended QR page; records also persisted
Visual forensics	Structural mismatch focus	SSIM, MSE, PSNR, FFT, DCT, edges, masks, regions
User interface	Prototype described in paper	React/Vite portal plus FastAPI API
Scope	PDF structural tamper detection	Visual Page 1 authentication plus DSP evidence

2.4 Research Gap and Project Contribution

Structural hashing can detect changes in PDF internals, but a user-facing workflow may need to answer a different question: has the visible first page changed? The project fills that engineering gap with a deterministic rasterization contract, a portable QR certificate, constant-time comparison, and visual explanations. This is an implementation contribution and integration achievement; the available evidence does not justify claiming a new cryptographic primitive or superiority over the base paper.

2.5 Related Approaches

The literature and search results also identify hybrid image-forgery approaches that combine cryptographic hashes with SSIM and OpenCV, and QR systems that embed document identifiers or digests. These approaches motivate the project's layered design, but their reported datasets and metrics are not reproduced here. They are listed as related context in the references rather than used to inflate the project's experimental claims.

Chapter 3: Theoretical Background

3.1 Canonical Rasterization

A PDF page is rendered at a fixed resolution r = 150 DPI. Since PDF coordinates use 72 points per inch, the renderer uses zoom z = r / 72. The page is rendered in RGB with alpha disabled. If P denotes the resulting ordered byte sequence, the stored digest is H = SHA-256(P). Equal canonical pixel sequences produce equal digests; any changed pixel changes the digest with the collision-resistance assumptions of SHA-256.

3.2 Cryptographic Hash Comparison

The verification service compares the QR-derived digest h_q and the newly computed digest h_c using a constant-time comparison. In abstract form, match = h_q == h_c. The implementation uses hmac.compare_digest rather than an ordinary early-exit string comparison. A match is authoritative for the canonical representation, while a mismatch is reported as MODIFIED unless the QR/page structure is invalid.

3.3 Similarity Measures

The DSP pipeline computes SSIM, MSE, and PSNR on preprocessed grayscale images. MSE is MSE = (1/N) Σ(x_i − y_i)^2. PSNR = 10 log10(MAX^2/MSE), where MAX is the peak sample value. SSIM compares local luminance, contrast, and structure; the implementation delegates image comparison to its similarity service and returns the resulting values.

3.4 Fourier-Domain Features

For a grayscale image x[m,n], the two-dimensional DFT is X[k,l] = Σ_m Σ_n x[m,n] exp(−j2π(km/M + ln/N)). The implementation computes a centered FFT using NumPy, forms log(1 + |X|), and extracts mean magnitude, standard deviation, total spectral energy, low-frequency energy, high-frequency energy, and their ratio. Feature similarity is the mean of bounded symmetric relative similarities.

3.5 DCT and Edge Features

The DCT concentrates spatial energy into low-frequency coefficients. The project uses OpenCV's two-dimensional DCT and compares DC, total, low-, mid-, and high-frequency energies. Sobel gradients and Canny edge maps provide structural features. Edge similarity uses Dice similarity: D = 2|A ∩ B|/(|A| + |B|), scaled to 0–100.

3.6 Tampering Localization

Candidate and reference images are normalized in size and aligned using phase-correlation translation, subject to a maximum shift. Absolute differences are thresholded and cleaned using opening, closing, dilation, and erosion. External contours above the minimum area become bounding boxes. This produces a mask and visual artifacts rather than replacing the cryptographic verdict.

Chapter 4: Proposed System Architecture and Methodology

4.1 Architecture

The system has a React/Vite client, a FastAPI service layer, PDF processing services, cryptographic and QR services, and an independent DSP forensic layer. The generate endpoint creates the sealed PDF and persists a document identifier, hash record, and canonical reference. The verify endpoint validates page count, computes the candidate hash, decodes the QR, compares digests, and optionally orchestrates DSP analysis.

1. User selects an original PDF in the Generate page.
2. Backend reads Page 1 and computes the canonical 150-DPI RGB SHA-256 digest.
3. A Level-H QR payload is generated and placed on a synthesized A4 Page 2.
4. The two-page output is serialized and returned with a download URL.
5. User uploads the secured file to Verify; the QR payload and candidate Page 1 are inspected.
6. The verdict is AUTHENTIC, MODIFIED, or INVALID; reference-backed requests additionally receive DSP scores and visual artifacts.

4.2 Storage and API Contracts

Endpoint	Function
GET /api/v1/health	Health status and version
POST /api/v1/generate/secure-pdf	Multipart PDF upload and secured-PDF generation
GET /api/v1/generate/download/{file}	Download generated output
POST /api/v1/verify/verify-pdf	Multipart secured-PDF verification; optional original_file compatibility path
GET /api/v1/visual-results/...	Serve generated forensic visual artifacts

4.3 DSP Configuration

Parameter	Configured value
Canonical resolution	150 DPI
SSIM threshold	0.95
Difference threshold	30
Minimum region area	100 pixels
FFT low-frequency ratio	0.10
DCT low/mid ratios	0.10 / 0.30
Canny thresholds	50 / 150
Maximum alignment shift	10.0
Tampering percentage threshold	5.0%
Authentication weights	Hash .40; SSIM .25; FFT .15; DCT .10; edge .10

4.4 Error States

A document with fewer than two pages is INVALID. A corrupt PDF or unreadable QR is also INVALID. A digest mismatch is MODIFIED in the primary verification service. The extended authentication engine can classify hash mismatches with rendering differences, suspicious confidence, or significant localized tampering; these classifications are supporting forensic telemetry and must not be confused with a cryptographic match.

Chapter 5: Implementation

5.1 Backend

The backend is organized as a FastAPI application with routers, Pydantic response schemas, configuration, utilities, and services. PyMuPDF performs rendering and PDF assembly. The hash service explicitly requests fitz.csRGB and alpha=False before hashing pixmap.samples. The PDF service copies only the original first page and appends a generated A4 certificate. The QR service creates and decodes the high-error-correction payload.

5.2 Verification and Security Properties

The verification service opens the PDF from bytes, checks page_count, hashes Page 1, decodes Page 2, extracts the digest and document identifier, and uses hmac.compare_digest. This design avoids trusting a user-provided calculated value and keeps hash comparison exact. It does not itself provide issuer authentication or prevent QR-page replacement by a malicious actor; those are deployment-level limitations.

5.3 Frontend

The frontend is a React 18 application using React Router, Axios, Vite, and Tailwind CSS. It exposes Home, Generate, Verify, and About pages plus reusable upload, loading, step-indicator, result, and dashboard components. Axios uses a configurable VITE_API_URL and a 60-second request timeout suitable for rendering and analysis.

5.4 Evidence Screenshots

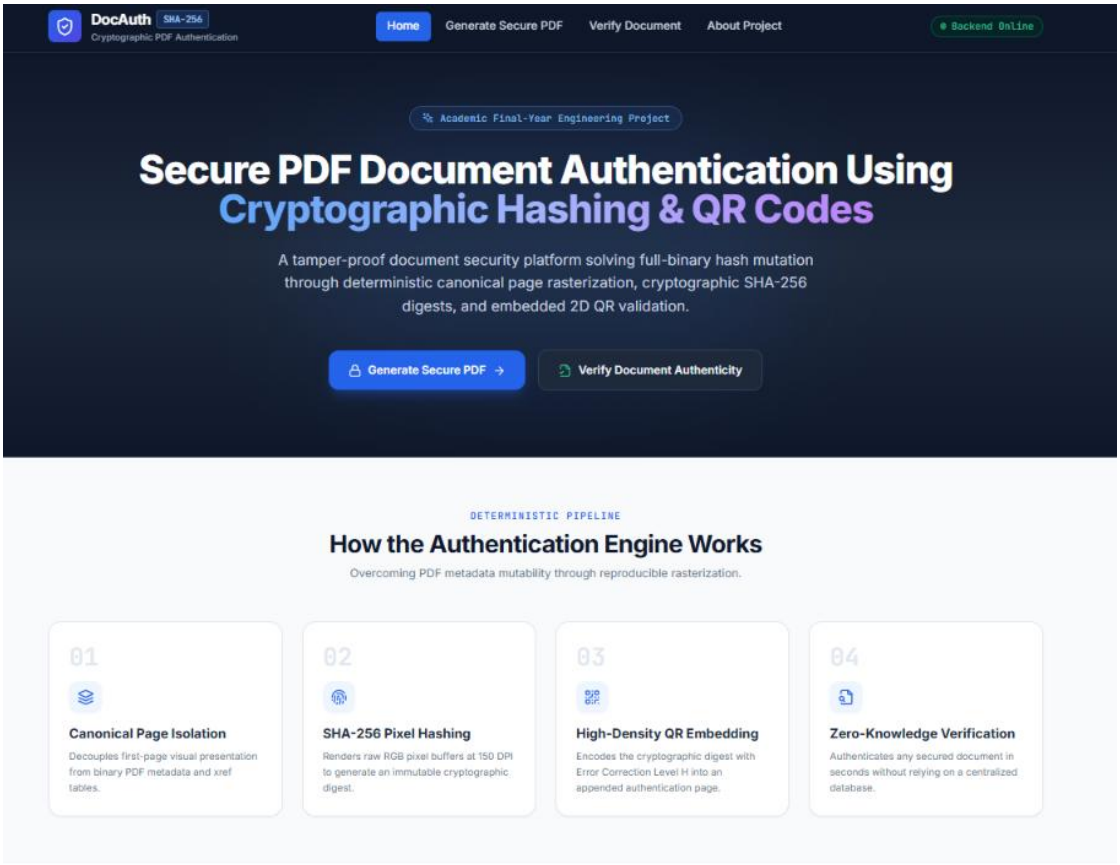


Figure 1. Implemented home page showing the portal's securing and verification workflow.

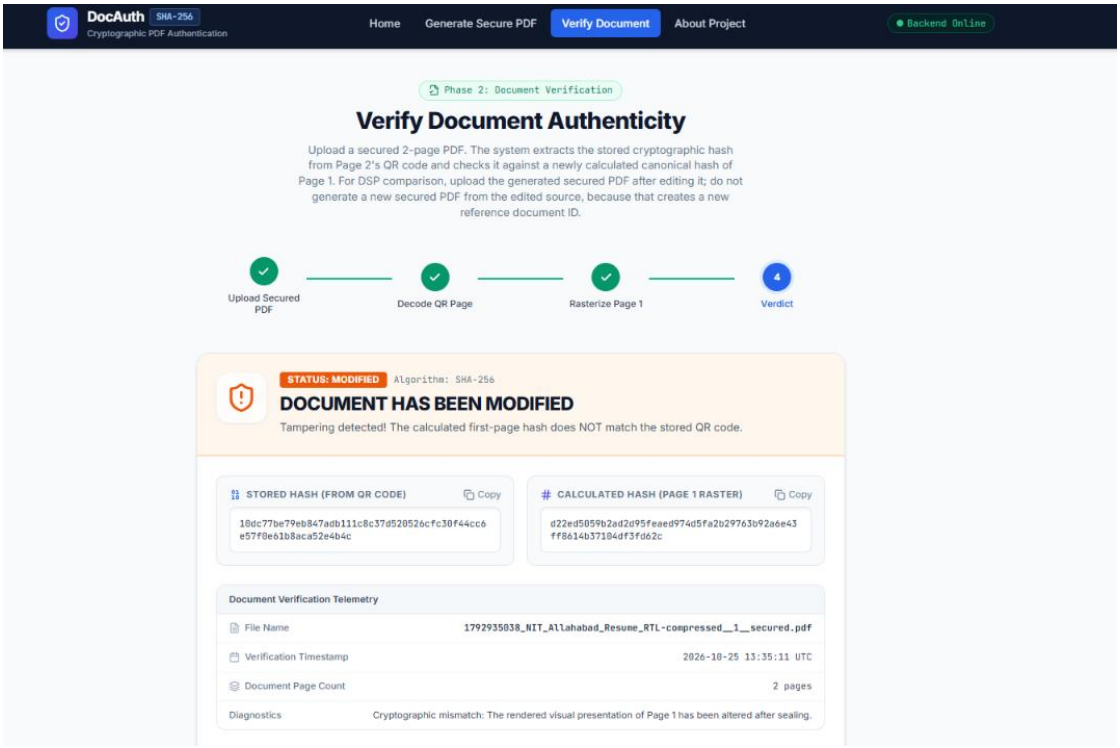


Figure 2. Supplied verification screenshot showing a MODIFIED verdict and hash mismatch telemetry.

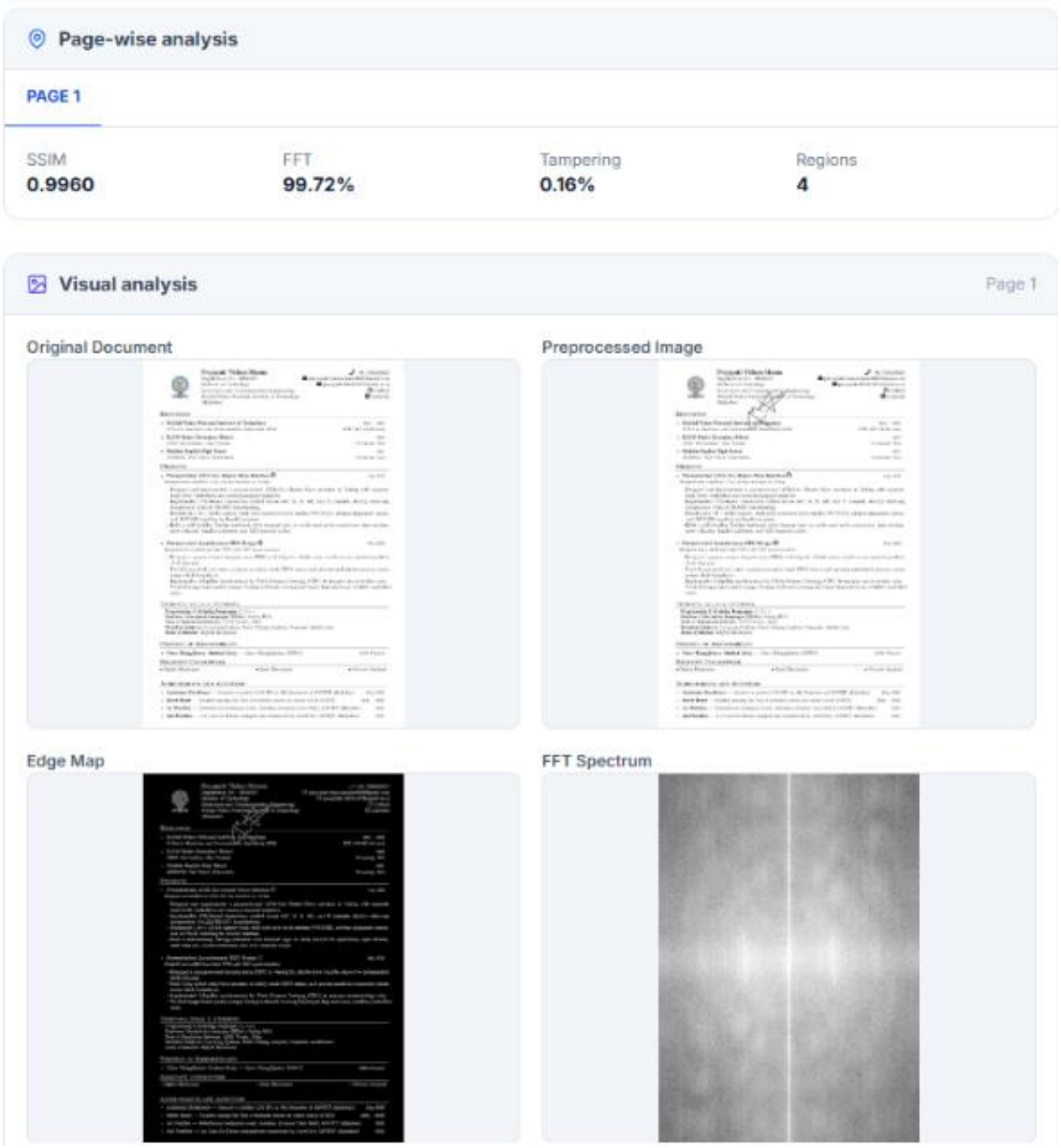


Figure 3. Supplied DSP dashboard screenshot showing SSIM, FFT, tampering percentage, regions, and visual artifacts.

Chapter 6: Experimental Evaluation and Results

6.1 Test Method

The repository's test_e2e.py creates a synthetic one-page transcript, calls the generate API, downloads the secured output, verifies it, modifies Page 1 by covering and replacing text, verifies the tampered file, and verifies the original one-page PDF as an invalid secured document. Unit tests cover FFT, DCT, feature extraction, preprocessing, similarity, tamper detection, authentication, and the verification API.

6.2 Verification Test Matrix

Case	Expected result	Evidence
Freshly secured PDF	AUTHENTIC; hashes match	test_e2e.py assertions and verification_service.py
Page 1 text altered	MODIFIED; hashes differ	test_e2e.py tamper step and supplied screenshot
Original one-page PDF	INVALID; page count < 2	test_e2e.py assertion
Identical DSP images	Similarity metrics at their maximum	test_similarity_service.py and test_fft_service.py
Different / modified images	Reduced frequency or similarity scores	test_fft_service.py and tamper tests

6.3 Observed Screenshot Values

One supplied screenshot reports SSIM 0.9960, FFT 99.72%, tampering 0.16%, and four detected regions for a modified document. These values are treated as an observed demonstration result, not as a population-level accuracy claim. The screenshot also shows the stored and calculated hashes differ, which is consistent with the exact cryptographic verdict.

6.4 Reproducibility and Evidence Limits

The code and tests support functional reproducibility. However, the repository does not include a controlled benchmark corpus, repeated trials, confusion matrix, latency distribution, false-positive/false-negative study, or comparison against the base paper's data. Therefore this report does not claim an accuracy percentage, superiority, or generalization to all PDF generators.

Chapter 7: Discussion, Limitations and Future Work

7.1 Strengths

- Clear separation between cryptographic authority and explanatory DSP telemetry.
- Deterministic RGB rendering reduces sensitivity to PDF metadata and serialization changes.
- QR certificate provides a portable digest transport mechanism.
- Visual outputs make localized changes inspectable by a human reviewer.
- Service-level tests cover both normal and failure paths.

7.2 Limitations

- Only the first page is canonically authenticated; later source pages are not retained in the sealed output.
- The QR digest is not digitally signed by an institutional private key.
- A copied or replaced QR certificate is not prevented without trusted records or signatures.
- Rendering can vary across library versions, fonts, color management, or platform behavior; the deployment must pin the environment.
- DSP scores depend on preprocessing, alignment, thresholds, and reference availability.
- No academic dataset or statistically powered benchmark is included.
- The selected base paper could not be verified from a complete supplied PDF; analysis relies on public indexed descriptions.

7.3 Future Work

- Add issuer signatures and certificate/key rotation for provenance.
- Define and version a canonical rendering profile including library versions, page box policy, fonts, and color handling.
- Support multi-page manifests while preserving the explicit first-page compatibility mode.
- Build a benchmark with synthetic and real PDF modifications and report ROC, precision, recall, F1, and latency.
- Evaluate structural page-object hashing jointly with visual hashing.
- Add adversarial tests for QR replacement, page insertion, scaling, rotation, and renderer differences.
- Provide an auditable database or transparency log for document identifiers.

Chapter 8: Conclusion

This project demonstrates a working end-to-end prototype for visual PDF authentication. Its core design replaces unstable full-binary hashing with a deterministic first-page rasterization contract, computes a SHA-256 digest over raw RGB pixels, transports that digest in a QR certificate, and verifies it using constant-time comparison. The additional DSP pipeline adds SSIM, MSE, PSNR, FFT, DCT, edge, and tamper-localization evidence for documents with a persisted reference.

The implementation is supported by source code, service tests, an end-to-end script, and supplied interface/results screenshots. Its main engineering value is the integration of exact integrity verification with interpretable forensic diagnostics. The correct academic conclusion is that the system is a completed prototype with demonstrable functional behavior; production-grade trust, cross-renderer invariance, and statistical performance remain future work.

References

[1] G. Grobler, S. Makura, and H. Venter, “A Technique for the Detection of PDF Tampering or Forgery,” arXiv preprint arXiv:2507.00827, 2025. https://arxiv.org/abs/2507.00827

[2] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015. https://doi.org/10.6028/NIST.FIPS.180-4

[3] A. K. Jain, Fundamentals of Digital Image Processing. Prentice Hall, 1989.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” IEEE Transactions on Image Processing, 13(4), 600–612, 2004. https://doi.org/10.1109/TIP.2003.819861

[5] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” Mathematics of Computation, 19(90), 297–301, 1965.

[6] S. W. Smith, The Scientist and Engineer's Guide to Digital Signal Processing, 2nd ed., California Technical Publishing, 1999.

[7] PyMuPDF documentation, PDF rendering and document manipulation. https://pymupdf.readthedocs.io/

[8] OpenCV documentation, image processing and QRCodeDetector APIs. https://docs.opencv.org/

[9] Project repository, Secure PDF Document Authentication. https://github.com/Pranay-Mathurkar/Secure-PDF-Document-Authentication

Appendix A: Repository Evidence Map

Claim	Evidence location
Canonical 150-DPI RGB SHA-256	backend/app/services/hash_service.py; backend/app/config.py
Two-page synthesis and QR certificate	backend/app/services/pdf_service.py; backend/app/services/qr_service.py
Primary verification verdict	backend/app/services/verification_service.py
FFT/DCT/edge/tamper analysis	backend/app/services/fft_service.py, dct_service.py, feature_service.py, tamper_detection_service.py
Weighted multi-layer authentication	backend/app/services/authentication_service.py
HTTP integration	backend/app/main.py; backend/app/routers/generate.py and verify.py
Automated functional workflow	test_e2e.py
Unit tests	backend/tests/test_*.py

Appendix B: Reproduction Commands

Backend (PowerShell):

```
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Frontend (PowerShell):

```
cd frontend
npm install
npm run dev
```

End-to-end check:

```
python test_e2e.py
```

Appendix C: Information Requiring Confirmation

- Supervisor name and designation.
- Exact title, venue, DOI, and complete text of the institution's required base paper, if different from the searched paper in Chapter 2.
- Final submission date and any institutional formatting rules not visible in the supplied reference screenshots.